

5

实现

Implementations

大多数情况下, 适当提出你的 `classes` (和 `class templates`) 定义以及 `functions` (和 `function templates`) 声明, 是花费最多心力的两件事。一旦正确完成它们, 相应的实现大多直截了当。尽管如此, 还是有些东西需要小心。太快定义变量可能造成效率上的拖延; 过度使用转型 (`casts`) 可能导致代码变慢又难维护, 又招来微妙难解的错误; 返回对象“内部数据之号码牌 (`handles`)”可能会破坏封装并留给客户虚吊号码牌 (`dangling handles`); 未考虑异常带来的冲击则可能导致资源泄漏和数据败坏; 过度热心地 `inlining` 可能引起代码膨胀; 过度耦合 (`coupling`) 则可能导致让人不满意的冗长建置时间 (`build times`)。

所有这些问题都可避免。本章逐一解释各种做法。

条款 26: 尽可能延后变量定义式的出现时间

Postpone variable definitions as long as possible.

只要你定义了一个变量而其类型带有一个构造函数或析构函数, 那么当程序的控制流 (`control flow`) 到达这个变量定义式时, 你便得承受构造成本; 当这个变量离开其作用域时, 你便得承受析构成本。即使这个变量最终并未被使用, 仍需耗费这些成本, 所以你应该尽可能避免这种情形。

或许你会认为, 你不可能定义一个不使用的变量, 但话不要说得太早! 考虑下面这个函数, 它计算通行密码的加密版本而后返回, 前提是密码够长。如果密码太短, 函数会丢出一个异常, 类型为 `logic_error` (定义于 C++ 标准程序库, 见条款 54):

```
//这个函数过早定义变量 "encrypted"
std::string encryptPassword(const std::string& password)
{
    using namespace std;
    string encrypted;
    if (password.length() < MinimumPasswordLength) {
        throw logic_error("Password is too short");
    }
    ...                //必要动作, 俾能将一个加密后的密码
                       //置入变量 encrypted 内
    return encrypted;
}
```

对象 `encrypted` 在此函数中并非完全未被使用, 但如果有个异常被丢出, 它就真的没被使用。也就是说如果函数 `encryptPassword` 丢出异常, 你仍得付出 `encrypted` 的构造成本和析构成本。所以最好延后 `encrypted` 的定义式, 直到确实需要它:

```
//这个函数延后 "encrypted" 的定义, 直到真正需要它
std::string encryptPassword(const std::string& password)
{
    using namespace std;
    if (password.length() < MinimumPasswordLength) {
        throw logic_error("Password is too short");
    }
    string encrypted;
    ...                //必要动作, 俾能将一个加密后的密码
                       //置入变量 encrypted 内
    return encrypted;
}
```

但是这段代码仍然不够秣纤合度, 因为 `encrypted` 虽获定义却无任何实参作为初值。这意味调用的是其 *default* 构造函数。许多时候你该对对象做的第一次事就是给它个值, 通常是通过一个赋值动作达成。条款 4 曾解释为什么“通过 *default* 构造函数构造出一个对象然后对它赋值”比“直接在构造时指定初值”效率差。那个分析当然也适用于此。举个例子, 假设 `encryptPassword` 的艰难部分在以下函数中进行:

```
void encrypt(std::string& s); //在其中的适当地点对 s 加密
```

于是 `encryptPassword` 可实现如下, 虽然还不算是最好的做法:

```
//这个函数延后 "encrypted" 的定义, 直到需要它为止。
//但此函数仍然有着不该有的效率低落。
std::string encryptPassword(const std::string& password)
{
    ...                //检查 length, 如前。
    std::string encrypted; //default-construct encrypted
    encrypted = password;  //赋值给 encrypted
    encrypt(encrypted);
    return encrypted;
}
```

更受欢迎的做法是以 `password` 作为 `encrypted` 的初值, 跳过毫无意义的 *default* 构造过程:

```
//终于, 这是定义并初始化 encrypted 的最佳做法
std::string encryptPassword(const std::string& password)
{
    ...                //检查长度。
    std::string encrypted(password); //通过 copy 构造函数
    //定义并初始化。

    encrypt(encrypted);
    return encrypted;
}
```

这让我们联想起本条款所谓“尽可能延后”的真正意义。你不只应该延后变量的定义, 直到非得使用该变量的前一刻为止, 甚至应该尝试延后这份定义直到能够给它初值实参为止。如果这样, 不仅能够避免构造(和析构)非必要对象, 还可以避免无意义的 *default* 构造行为。更深一层说, 以“具明显意义之初值”将变量初始化, 还可以附带说明变量的目的。

“但循环怎么办?” 你可能会感到疑惑。如果变量只在循环内使用, 那么把它定义于循环外并在每次循环迭代时赋值给它比较好, 还是该把它定义于循环内? 也就是说下面左右两个一般性结构, 哪一个比较好?

<pre>//方法 A: 定义于循环外 Widget w; for (int i = 0; i < n; ++i) { w = 取决于 i 的某个值; ... }</pre>	<pre>//方法 B: 定义于循环内 for (int i = 0; i < n; ++i) { Widget w(取决于 i 的某个值); ... }</pre>
--	--

这里我把对象的类型从 `string` 改为 `Widget`，以免造成读者对于“对象执行构造、析构、或赋值动作所需的成本”有任何特殊偏见。

在 `Widget` 函数内部，以上两种写法的成本如下：

- 做法 A：1 个构造函数 + 1 个析构函数 + n 个赋值操作
- 做法 B： n 个构造函数 + n 个析构函数

如果 `classes` 的一个赋值成本低于一组构造+析构成本，做法 A 大体而言比较高效。尤其当 n 值很大的时候。否则做法 B 或许较好。此外做法 A 造成名称 `w` 的作用域（覆盖整个循环）比做法 B 更大，有时那对程序的可理解性和易维护性造成冲突。因此除非 (1) 你知道赋值成本比“构造+析构”成本低，(2) 你正在处理代码中效率高度敏感（*performance-sensitive*）的部分，否则你应该使用做法 B。

请记住

- 尽可能延后变量定义式的出现。这样做可增加程序的清晰度并改善程序效率。

条款 27：尽量少做转型动作

Minimize casting.

C++ 规则的设计目标之一是，保证“类型错误”绝不可能发生。理论上如果你的程序很“干净地”通过编译，就表示它并不企图在任何对象身上执行任何不安全、无意义、愚蠢荒谬的操作。这是一个极具价值的保证，可别草率地放弃它。

不幸的是，转型（*casts*）破坏了类型系统（*type system*）。那可能导致任何种类的麻烦，有些容易辨识，有些非常隐晦。如果你来自 C, Java 或 C# 阵营，请特别注意，因为那些语言中的转型（*casting*）比较必要而无法避免，也比较不危险（与 C++ 相较）。但 C++ 不是 C，也不是 Java 或 C#。在 C++ 中转型是一个你会想着极大尊重去亲近的一个特性。

让我们首先回顾转型语法，因为通常有三种不同的形式，可写出相同的转型动作。C 风格的转型动作看起来像这样：

```
(T) expression           //将 expression 转型为 T
```

函数风格的转型动作看起来像这样：

```
T(expression)           //将 expression 转型为 T
```

两种形式并无差别, 纯粹只是小括号的摆放位置不同而已。我称此二种形式为“旧式转型” (*old-style casts*)。

C++ 还提供四种新式转型 (常常被称为 *new-style* 或 *C++-style casts*) :

```
const_cast<T>( expression )
dynamic_cast<T>( expression )
reinterpret_cast<T>( expression )
static_cast<T>( expression )
```

各有不同的目的:

- `const_cast` 通常被用来将对象的常量性转除 (cast away the constness)。它也是唯一有此能力的 C++-style 转型操作符。
- `dynamic_cast` 主要用来执行“安全向下转型” (safe downcasting), 也就是用来决定某对象是否归属继承体系中的某个类型。它是唯一无法由旧式语法执行的动作, 也是唯一可能耗费重大运行成本的转型动作 (稍后细谈)。
- `reinterpret_cast` 意图执行低级转型, 实际动作 (及结果) 可能取决于编译器, 这也就表示它不可移植。例如将一个 *pointer to int* 转型为一个 `int`。这一类转型在低级代码以外很少见。本书只使用一次, 那是在讨论如何针对原始内存 (raw memory) 写出一个调试用的分配器 (debugging allocator) 时, 见条款 50。
- `static_cast` 用来强迫隐式转换 (implicit conversions), 例如将 `non-const` 对象转为 `const` 对象 (就像条款 3 所为), 或将 `int` 转为 `double` 等等。它也可以用来执行上述多种转换的反向转换, 例如将 `void*` 指针转为 `typed` 指针, 将 *pointer-to-base* 转为 *pointer-to-derived*。但它无法将 `const` 转为 `non-const`——这个只有 `const_cast` 才办得到。

旧式转型仍然合法, 但新式转型较受欢迎。原因是: 第一, 它们很容易在代码中被辨识出来 (不论是人工辨识或使用工具如 `grep`), 因而得以简化“找出类型系统在哪个地点被破坏”的过程。第二, 各转型动作的目标愈窄化, 编译器愈可能诊断出错误的运用。举个例子, 如果你打算将常量性 (constness) 去掉, 除非使用新式转型中的 `const_cast` 否则无法通过编译。

我唯一使用旧式转型的时机是, 当我要调用一个 `explicit` 构造函数将一个对象传递给一个函数时。例如: